Name-Krushna Yerawar

PRN-202301070100

Batch-T4

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.optimizers import Adam, SGD
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import classification_report, confusion_matrix
```

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
```

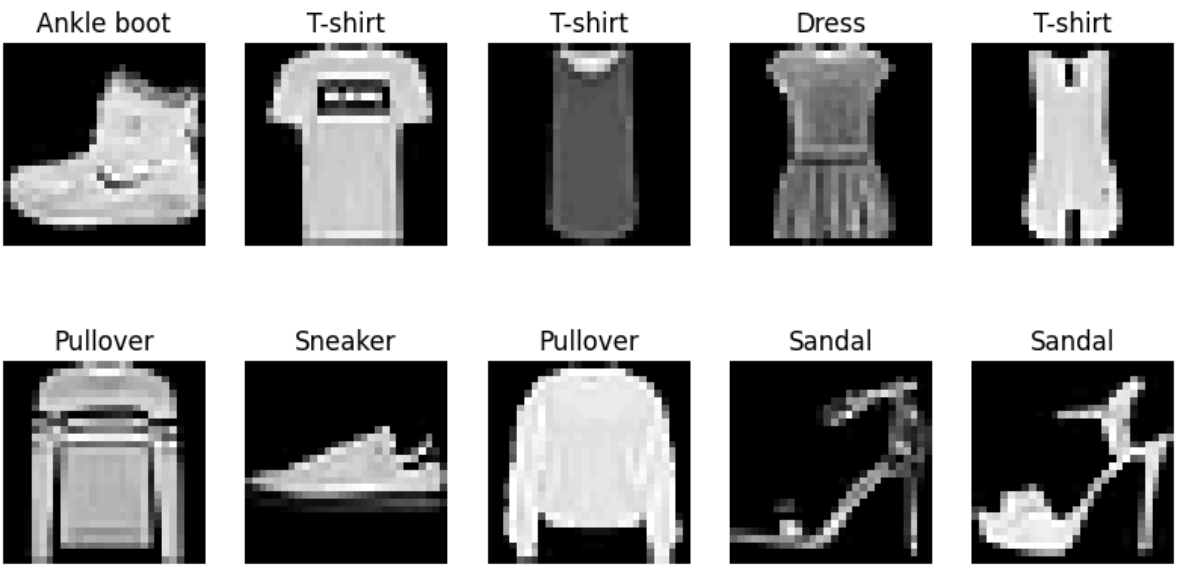
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-data
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
```

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

```
y_train_cat = to_categorical(y_train, num_classes=10)
y_test_cat = to_categorical(y_test, num_classes=10)
```

```
class_names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

plt.figure(figsize=(10,5))
for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(x_train[i], cmap='gray')
    plt.title(class_names[y_train[i]])
    plt.axis('off')
plt.show()
```




```
model = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(256, activation='relu'),
    Dropout(0.3),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py
  super().__init__(**kwargs)
```

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 256)	200,960
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
dense_2 (Dense)	(None, 10)	1,290

Total params: 235,146 (918.54 KB)
Trainable params: 235,146 (918.54 KB)
Non-trainable params: 0 (0.00 B)

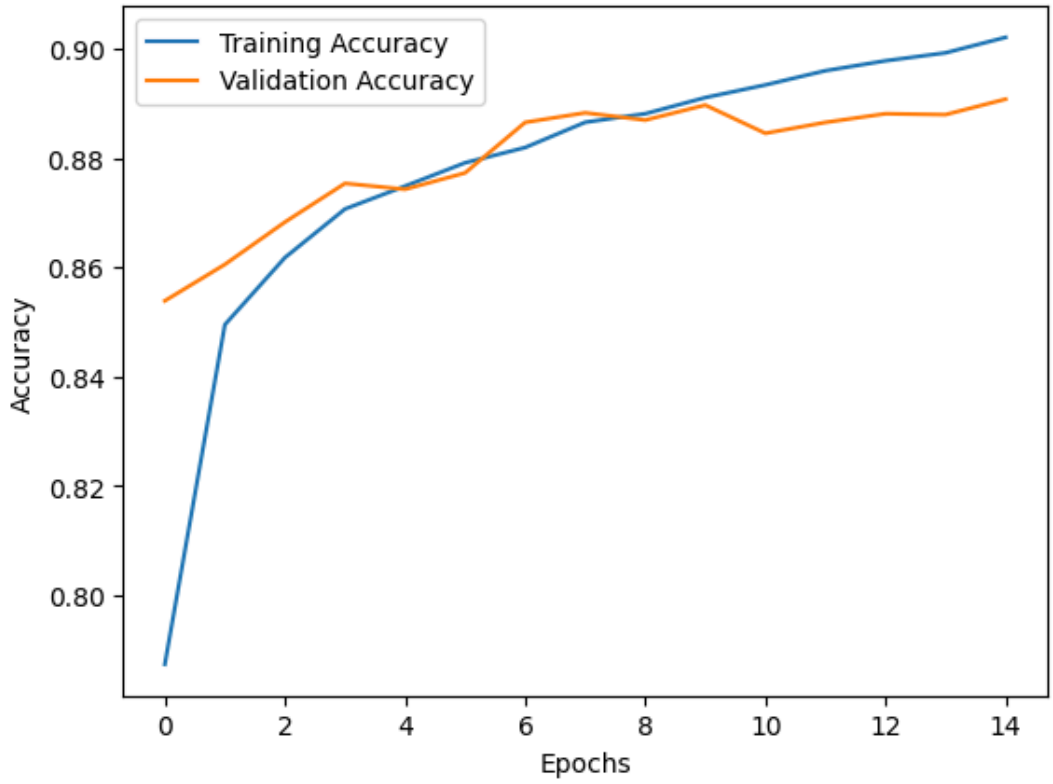
```
history = model.fit(
    x_train,
    y_train_cat,
    epochs=15,
    batch_size=128,
    validation_split=0.2
)
```

```
Epoch 1/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - accuracy: 0.7125 - loss: 0.8034 -
Epoch 2/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.8493 - loss: 0.4159 -
Epoch 3/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - accuracy: 0.8599 - loss: 0.3817 -
Epoch 4/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.8709 - loss: 0.3503 -
Epoch 5/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.8763 - loss: 0.3343 -
Epoch 6/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 6s 14ms/step - accuracy: 0.8810 - loss: 0.3259 -
Epoch 7/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.8812 - loss: 0.3172 -
Epoch 8/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.8893 - loss: 0.2995 -
Epoch 9/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.8896 - loss: 0.2953 -
Epoch 10/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.8930 - loss: 0.2879 -
Epoch 11/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.8914 - loss: 0.2854 -
Epoch 12/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.8945 - loss: 0.2833 -
Epoch 13/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.8987 - loss: 0.2679 -
Epoch 14/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 10ms/step - accuracy: 0.8993 - loss: 0.2615 -
Epoch 15/15
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9010 - loss: 0.2641 -
```

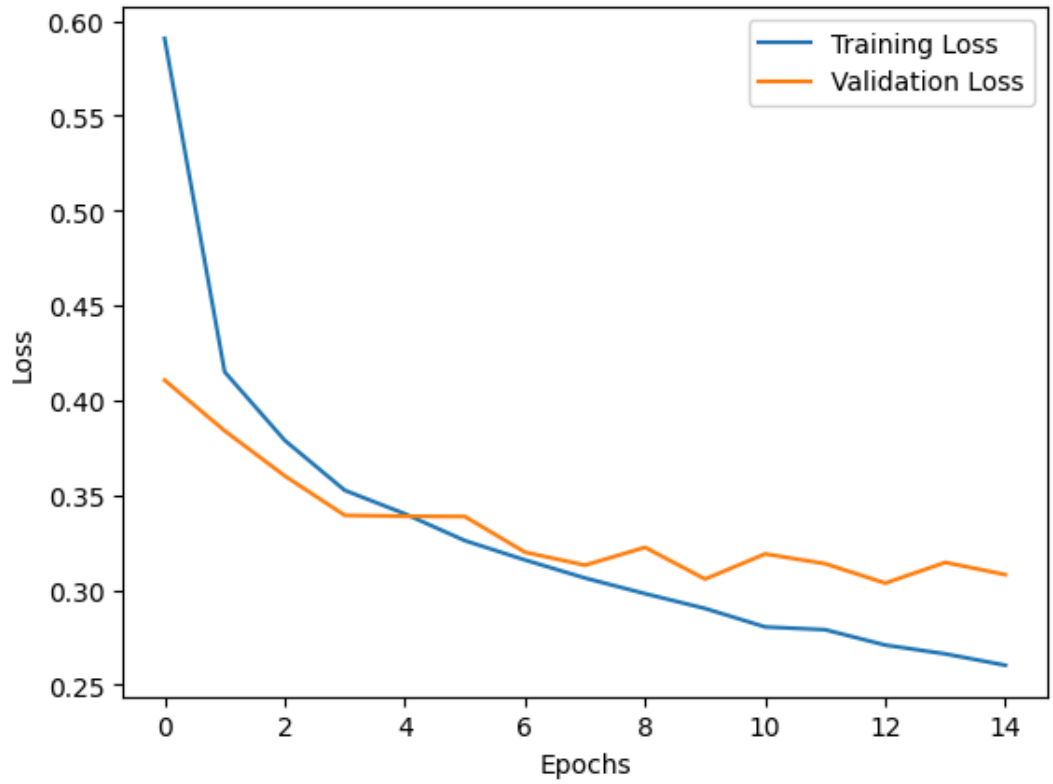
```
test_loss, test_accuracy = model.evaluate(x_test, y_test_cat)
print("Test Accuracy:", test_accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8831 - loss: 0.3303
Test Accuracy: 0.8845999836921692
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```




```
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```




```
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
```

```
print("Classification Report:")
print(classification_report(y_test, y_pred_classes))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.86      0.84      1000
           1       0.99      0.97      0.98      1000
           2       0.76      0.84      0.80      1000
           3       0.87      0.91      0.89      1000
           4       0.80      0.80      0.80      1000
           5       0.97      0.97      0.97      1000
           6       0.75      0.61      0.68      1000
           7       0.94      0.96      0.95      1000
           8       0.97      0.97      0.97      1000
           9       0.96      0.95      0.96      1000

    accuracy                           0.88     10000
   macro avg       0.88      0.88      0.88     10000
weighted avg       0.88      0.88      0.88     10000
```

```
cm = confusion_matrix(y_test, y_pred_classes)
print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[864   1  13  19   3   0  93   0   7   0]
 [  2 969   1  23   2   0   2   0   1   0]
 [ 17   2 844  15  79   0  43   0   0   0]
 [ 22   9  10 909  26   0  20   0   4   0]
 [  1   0 128  36 795   0  38   0   2   0]
 [  0   0   0   1   0 968   0  21   2   8]
 [143   0 112  35  84   0 615   0  11   0]
 [  0   0   0   0   0  14   0 958   1  27]
 [  6   0   1   3   4   3   4   4 975   0]
 [  0   0   0   0   0  12   1  38   0 949]]
```

```
model_sgd = Sequential([
    Flatten(input_shape=(28,28)),
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model_sgd.compile(
    optimizer=SGD(learning_rate=0.01, momentum=0.9),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model_sgd.fit(
    x_train,
    y_train_cat,
    epochs=10,
    batch_size=128,
```

```
    validation_split=0.2
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py
  super().__init__(**kwargs)
Epoch 1/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 7s 13ms/step - accuracy: 0.6841 - loss: 0.9427 -
Epoch 2/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.8392 - loss: 0.4562 -
Epoch 3/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.8570 - loss: 0.4075 -
Epoch 4/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 6s 16ms/step - accuracy: 0.8661 - loss: 0.3747 -
Epoch 5/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.8713 - loss: 0.3548 -
Epoch 6/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.8819 - loss: 0.3309 -
Epoch 7/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 6s 17ms/step - accuracy: 0.8854 - loss: 0.3223 -
Epoch 8/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.8860 - loss: 0.3139 -
Epoch 9/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.8899 - loss: 0.3003 -
Epoch 10/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.8947 - loss: 0.2892 -
<keras.src.callbacks.history.History at 0x7c058ff5c860>
```

```
model.save("fashion_mnist_model.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or
```

```
loaded_model = tf.keras.models.load_model("fashion_mnist_model.h5")
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to
```